

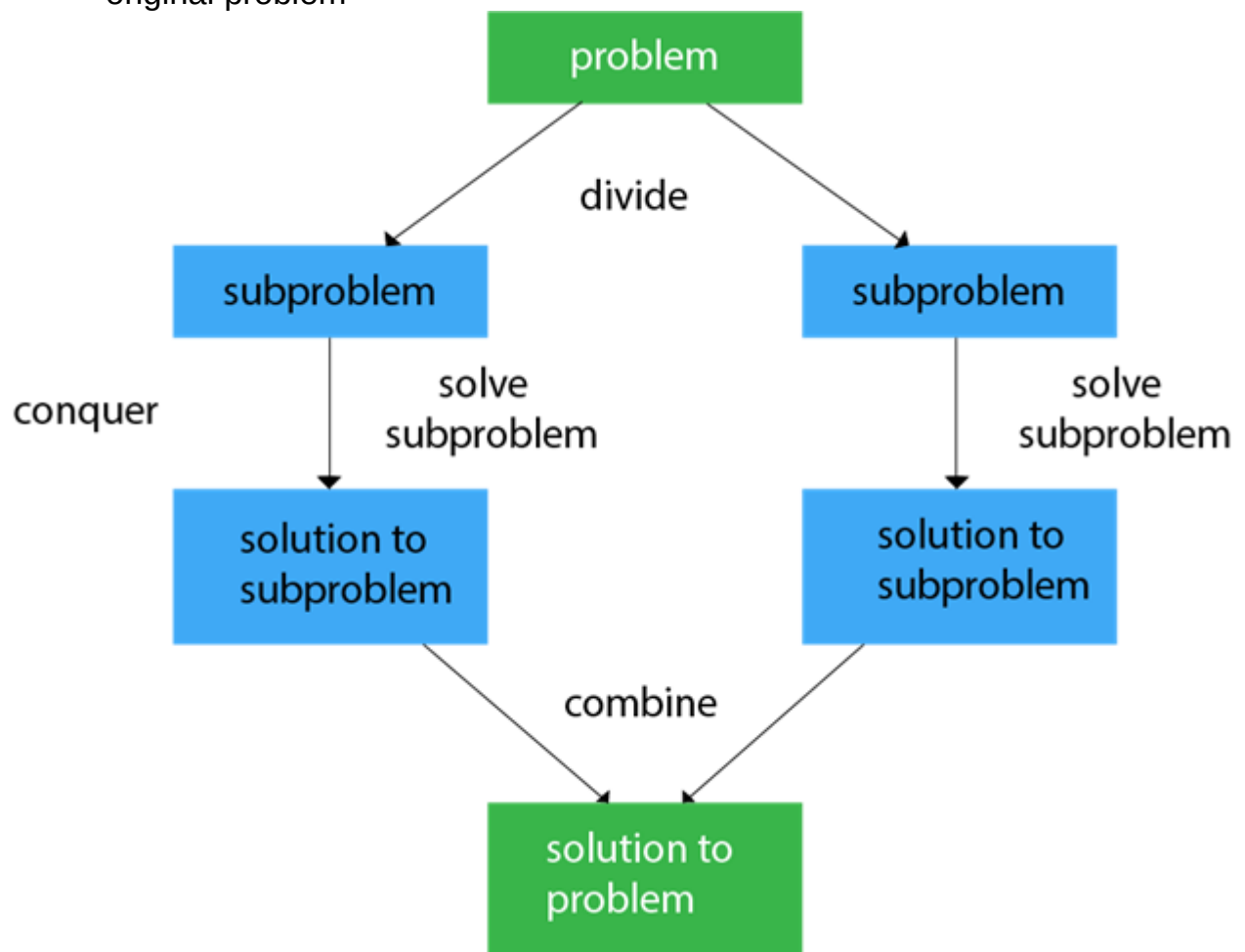
Unit 3 Notes

Divide and conquer

In divide and conquer approach, a problem is divided into smaller problems, then the smaller problems are solved independently, and finally the solutions of smaller problems are combined into a solution for the large problem.

Generally, divide-and-conquer algorithms have three parts –

- Divide the problem into a number of subproblems that are smaller instances of the same problem.
- Conquer the sub-problems by solving them recursively. If they are small enough, solve the sub-problems as base cases.
- Combine the solutions to the sub-problems into the solution for the original problem



Application of Divide and Conquer Approach

Following are some problems, which are solved using divide and conquer approach.

- Finding the maximum and minimum of a sequence of numbers
- Strassen's matrix multiplication
- Merge sort
- Binary search

Matrix Multiplication

Naive Method

Naive matrix multiplication refers to the naive algorithm for executing **matrix multiplication**: we calculate each entry as the sum of products.

Explicitly, suppose

$$A = (a_{ij})_{1 \leq i \leq m, 1 \leq j \leq n}$$

is a $m \times n$ matrix and

$$B = (b_{jk})_{1 \leq j \leq n, 1 \leq k \leq p}$$

is a $n \times p$ matrix, and denote by

$$C = (c_{ik})_{1 \leq i \leq m, 1 \leq k \leq p}$$

the product of the matrices. We then have the following formula:

$$c_{ik} = \sum_{j=1}^n a_{ij} b_{jk}$$

Code for Naive Method

```
void multiply(int A[][N], int B[][N], int C[][N])
{
    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
        {
            C[i][j] = 0;
            for (int k = 0; k < N; k++)
```

```
        {  
            C[i][j] += A[i][k]*B[k][j];  
        }  
    }  
}
```

Time Complexity of above method is **$O(N^3)$** .

Using DAC

Divide & Conquer:

Partition A , B , and C into four $n/2 \times n/2$ matrices:

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}.$$

Hence the equation $C = A \cdot B$ becomes:

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \cdot \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

This corresponds to the four equations:

$$C_{11} = A_{11} \cdot B_{11} + A_{12} \cdot B_{21}$$

$$C_{12} = A_{11} \cdot B_{12} + A_{12} \cdot B_{22}$$

$$C_{21} = A_{21} \cdot B_{11} + A_{22} \cdot B_{21}$$

$$C_{22} = A_{21} \cdot B_{12} + A_{22} \cdot B_{22}$$

SQUARE-MATRIX-MULTIPLY-RECURSIVE(A, B)

```
1   $n = A.rows$ 
2  let  $C$  be a new  $n \times n$  matrix
3  if  $n == 1$ 
4       $c_{11} = a_{11} \cdot b_{11}$ 
5  else partition  $A$ ,  $B$ , and  $C$  as in equations (4.9)
6       $C_{11} = \text{SQUARE-MATRIX-MULTIPLY-RECURSIVE}(A_{11}, B_{11})$ 
           +  $\text{SQUARE-MATRIX-MULTIPLY-RECURSIVE}(A_{12}, B_{21})$ 
7       $C_{12} = \text{SQUARE-MATRIX-MULTIPLY-RECURSIVE}(A_{11}, B_{12})$ 
           +  $\text{SQUARE-MATRIX-MULTIPLY-RECURSIVE}(A_{12}, B_{22})$ 
8       $C_{21} = \text{SQUARE-MATRIX-MULTIPLY-RECURSIVE}(A_{21}, B_{11})$ 
           +  $\text{SQUARE-MATRIX-MULTIPLY-RECURSIVE}(A_{22}, B_{21})$ 
9       $C_{22} = \text{SQUARE-MATRIX-MULTIPLY-RECURSIVE}(A_{21}, B_{12})$ 
           +  $\text{SQUARE-MATRIX-MULTIPLY-RECURSIVE}(A_{22}, B_{22})$ 
10 return  $C$ 
```

Let $T(n)$ be the runtime of this procedure. Then:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ 8 \cdot T(n/2) + \Theta(n^2) & \text{if } n > 1. \end{cases}$$

Solution: $T(n) = \Theta(8^{\log_2 n}) = \Theta(n^3)$

Strassen's Algo

• **Strassen's Algorithm (1969)**

Partition each of the matrices into four $n/2 \times n/2$ submatrices

Create 10 matrices $S_1, S_2, \dots, S_{10}$. Each is $n/2 \times n/2$ and is the sum or difference of two matrices created in the previous step.

Recursively compute 7 matrix products $P_1, P_2, \dots, P_7$, each $n/2 \times n/2$

Compute $n/2 \times n/2$ submatrices of C by adding and subtracting various combinations of the P_i .

The 10 Submatrices and 7 Products

$$P_1 = A_{11} \cdot S_1 = A_{11} \cdot (B_{12} - B_{22})$$

$$P_2 = S_2 \cdot B_{22} = (A_{11} + A_{12}) \cdot B_{22}$$

$$P_3 = S_3 \cdot B_{11} = (A_{21} + A_{22}) \cdot B_{11}$$

$$P_4 = A_{22} \cdot S_4 = A_{22} \cdot (B_{21} - B_{11})$$

$$P_5 = S_5 \cdot S_6 = (A_{11} + A_{22}) \cdot (B_{11} + B_{22})$$

$$P_6 = S_7 \cdot S_8 = (A_{12} - A_{22}) \cdot (B_{21} + B_{22})$$

$$P_7 = S_9 \cdot S_{10} = (A_{11} - A_{21}) \cdot (B_{11} + B_{12})$$

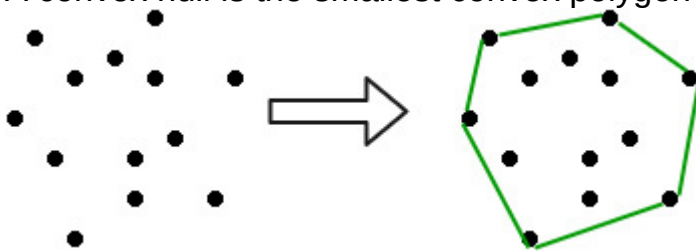
Claim

$$\begin{pmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{21} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{pmatrix} = \begin{pmatrix} P_5 + P_4 - P_2 + P_6 & P_1 + P_2 \\ P_3 + P_4 & P_5 + P_1 - P_3 - P_7 \end{pmatrix}$$

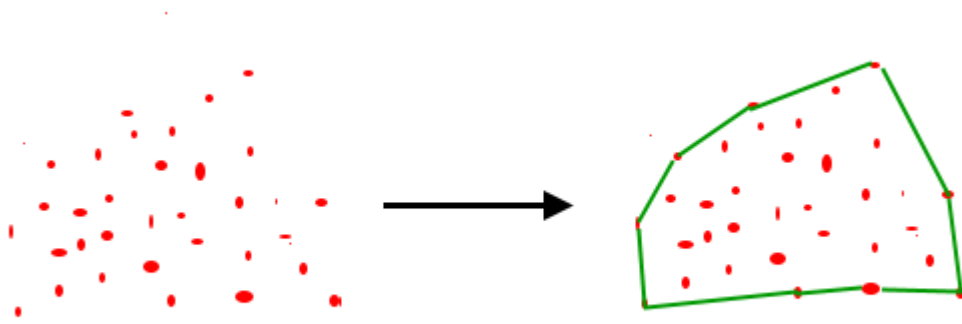
NOTE: Algo for Matrix Multiplications	Time Complexity (T.C)	Space Complexity (S.C)
* NAIVE METHOD (without using DAC)	$O(n^3)$	NIL
* Using DAC	$O(n^3)$	$O(\log n)$
* Strassen's Algo (using DAC)	$O(n^{2.81})$	Little bit extra than using DAC S.C

Convex Hull

A convex hull is the smallest convex polygon containing all the given points.



Graham Scan Algorithm:



Using Graham's scan algorithm, we can find Convex Hull in $O(n \log n)$ time.

Algorithm:

- 1) Find the bottom-most point by comparing y coordinate of all points. If there are two points with the same y value, then the point with smaller x coordinate value is considered. Let the bottom-most point be P_0 . Put P_0 at first position in the output hull.
- 2) Consider the remaining $n-1$ points and sort them by polar angle in counterclockwise order around $\text{points}[0]$. If the polar angle of two points is the same, then put the nearest point first.
- 3 After sorting, check if two or more points have the same angle. If two more points have the same angle, then remove all same angle points except the point farthest from P_0 . Let the size of the new array be m .
- 4) If m is less than 3, return (Convex Hull not possible)
- 5) Create an empty stack 'S' and push $\text{points}[0]$, $\text{points}[1]$ and $\text{points}[2]$ to S.
- 6) Process remaining $m-3$ points one by one. Do following for every point ' $\text{points}[i]$ '
 - 4.1) Keep removing points from the stack while orientation of following 3 points is not counterclockwise (or they don't make a left turn).
 - a) Point next to top in stack
 - b) Point at the top of stack
 - c) $\text{points}[i]$
 - 4.2) Push $\text{points}[i]$ to S
- 5) Print contents of S

Time Complexity:

The merging of the left and the right convex hulls take $O(n)$ time and as we are dividing the points into two equal parts, so the time complexity of the above algorithm is $O(n * \log n)$.

Quick Hull

Quickhull is a method of computing the **convex hull** of a finite set of points in the plane. It uses a divide and conquer approach similar to that of quicksort, from which its name derives. Its average case complexity is considered to be $\Theta(n * \log(n))$, whereas in the worst case it takes $O(n^2)$.

When a convex Hull H of a set of points S is known, then the convex Hull H_1 of the set of points S_1 , that is $S +$ a new point P , is computed as follows:

- Let P_1 and P_2 be the closest point to P in the left and right section respectively
- Remove line P_1 to P_2 from convex Hull H
- Add lines P_1 to P and P to P_2 to modified convex hull H to get new convex Hull H_1

Quick hull is a method of constructing a smallest convex polygon out of n given points in a plane

1. It is proved that the quick hull algorithm runs faster if the input uses non-extreme points and also, if it uses less memory.
2. Eddy formulated a quick hull algorithm.
3. Quick hull problem and closest pair algorithms are some of the examples of computational problems.
4. The worst case complexity of quickhull algorithm using divide and conquer approach is mathematically found to be $O(N^2)$.

Greedy Methods

A **greedy algorithm** is a simple, intuitive algorithm that is used in optimization problems. The algorithm makes the optimal choice at each step as it attempts to find the overall optimal way to solve the entire problem. However, in many problems, a greedy strategy does not produce an optimal solution.

A greedy algorithm, as the name suggests, always makes the choice that seems to be the best at that moment. This means that it makes a locally-optimal choice in the hope that this choice will lead to a globally-optimal solution.

Key Features:

1. Greedy algorithms build a solution part by part, choosing the next part in such a way, that it gives an immediate benefit.
2. This approach never reconsiders the choices taken previously.
3. This approach is mainly used to solve optimization problems.

The greedy algorithm consists of four (4) functions.

1. A function that checks whether a chosen set of items provide a solution.
2. A function that checks the feasibility of a set.
3. The selection function tells which of the candidates is the most promising.
4. An objective function, which does not appear explicitly, gives the value of a solution.

Applications of Greedy Method

1. Finding the shortest path between two vertices using Dijkstra's algorithm.
2. Finding the minimal spanning tree in a graph using Prim's /Kruskal's algorithm, etc

Where Greedy Approach Fails

In many problems, Greedy algorithms fail to find an optimal solution, moreover it may produce a worst solution. Problems like Travelling Salesman and Knapsack cannot be solved using this approach.

Characteristics and Phases of Greedy Method

Greedy Method finds out of many options, but you have to choose the best option.

The greedy algorithm consists of four (4) function.

1. A function that checks whether chosen set of items provide a solution.
2. A function that checks the feasibility of a set.

3. The selection function tells which of the candidates is the most promising.
4. An objective function, which does not appear explicitly, gives the value of a solution

Dynamic programming VS Greedy Method

FEATURE	GREEDY METHOD	DYNAMIC PROGRAMMING
Feasibility	In a greedy Algorithm, we make whatever choice seems best at the moment in the hope that it will lead to global optimal solution.	In Dynamic Programming we make decision at each step considering current problem and solution to previously solved sub problem to calculate optimal solution .
Optimality	In Greedy Method, sometimes there is no such guarantee of getting Optimal Solution.	It is guaranteed that Dynamic Programming will generate an optimal solution as it generally considers all possible cases and then choose the best.

Recursion	A greedy method follows the problem solving heuristic of making the locally optimal choice at each stage.	A Dynamic programming is an algorithmic technique which is usually based on a recurrent formula that uses some previously calculated states.
Memorization	It is more efficient in terms of memory as it never look back or revise previous choices	It requires dp table for memorization and it increases it's memory complexity.
Time complexity	Greedy methods are generally faster. For example, Dijkstra's shortest path algorithm takes $O(E\log V + V\log V)$ time.	Dynamic Programming is generally slower. For example, Bellman Ford algorithm takes $O(VE)$ time.

Fashion	The greedy method computes its solution by making its choices in a serial forward fashion, never looking back or revising previous choices.	Dynamic programming computes its solution bottom up or top down by synthesizing them from smaller optimal sub solutions.
Example	Fractional knapsack .	0/1 knapsack problem

KnapSack Problem

Knapsack basically means bag. A bag of given capacity.

We want to pack n items in your luggage.

- The i th item is worth v_i dollars and weight w_i pounds.
- Take as valuable a load as possible, but cannot exceed W pounds.
- v_i, w_i, W are integers.

$$W \leq \text{capacity}$$

$$\text{Value} \leftarrow \text{Max}$$

- Knapsack of capacity
- List (Array) of weight and their corresponding value.

KNAPSACK (n, W)

```

1. for  $w = 0, W$ 
2. do  $V[0, w] \leftarrow 0$ 
3. for  $i = 0, n$ 
4. do  $V[i, 0] \leftarrow 0$ 
5. for  $w = 0, W$ 
6. do if ( $w_i \leq w$  &  $v_i + V[i-1, w - w_i] > V[i-1, w]$ )

```

```

7. then  $V[i, W] \leftarrow v_i + V[i - 1, w - w_i]$ 
8. else  $V[i, W] \leftarrow V[i - 1, w]$ 

```

0-1 Knapsack Problem

This item cannot be broken which means the thief should take the item as a whole or should leave it. That's why it is called the 0/1 knapsack Problem.

- Each item is taken or not taken.
- Cannot take a fractional amount of an item taken or take an item more than once.
- It cannot be solved by the Greedy Approach because it is enabled to fill the knapsack to capacity.
- Greedy Approach doesn't ensure an Optimal Solution.

Fractional Knapsack Problem

In this case, items can be broken into smaller pieces, hence the thief can select fractions of items.

According to the problem statement,

- There are n items in the store
- Weight of i th item $w_i > 0$
- Profit for i th item $p_i > 0$ and
- Capacity of the Knapsack is W

In this version of Knapsack problem, items can be broken into smaller pieces. So, the thief may take only a fraction x_i of **i th** item.

$$0 \leq x_i \leq 1$$

The i th item contributes the weight $x_i \cdot w_i$ to the total weight in the knapsack and profit $x_i \cdot p_i$ to the total profit.

Hence, the objective of this algorithm is to

$$\text{maximize } \sum_{i=1}^n (x_i \cdot p_i)$$

subject to constraint,

$$\sum_{n=1}^n (x_i \cdot w_i) \leq W$$

It is clear that an optimal solution must fill the knapsack exactly, otherwise we could add a fraction of one of the remaining items and increase the overall profit.

Thus, an optimal solution can be obtained by

$$\sum_{n=1}^n (x_i \cdot w_i) = W$$

Therefore, the total time including the sort is in **$O(n \log n)$** .

Optimal Knapsack Algo for Fractional KnapSack

Algorithm:

```
Greedy-Fractional-Knapsack (w[1..n], p[1..n], W)
for i = 1 to n
    do x[i] = 0
weight = 0
for i = 1 to n
    if weight + w[i] ≤ W then
        x[i] = 1
        weight = weight + w[i]
    else
        x[i] = (W - weight) / w[i]
        weight = W
        break
return x
```

Example

Let us consider that the capacity of the knapsack $W = 60$ and the list of provided items are shown in the following table –

Item	A	B	C	D
Profit	280	100	120	120
Weight	40	10	20	24
Ratio ($\frac{p_i}{w_i}$)	7	10	6	5

As the provided items are not sorted based on $\frac{p_i}{w_i}$. After sorting, the items are as shown in the following table.

Item	B	A	C	D
Profit	100	280	120	120
Weight	10	40	20	24
Ratio ($\frac{p_i}{w_i}$)	10	7	6	5

Solution

After sorting all the items according to $\frac{p_i}{w_i}$. First all of **B** is chosen as weight of **B** is less than the capacity of the knapsack.

Next, item **A** is chosen, as the available capacity of the knapsack is greater than the weight of **A**. Now, **C** is chosen as the next item. However, the whole item cannot be chosen as the remaining capacity of the knapsack is less than the weight of **C**.

Hence, fraction of **C** (i.e. $(60 - 50)/20$) is chosen.

Now, the capacity of the Knapsack is equal to the selected items. Hence, no more item can be selected.

The total weight of the selected items is $10 + 40 + 20 * (10/20) = 60$

And the total profit is $100 + 280 + 120 * (10/20) = 380 + 60 = 440$

This is the optimal solution. We cannot gain more profit selecting any different combination of items.

Optimal Knapsack Algo for 0-1 KnapSack

```
Dynamic-0-1-knapsack (v, w, n, W)
```

```
for w = 0 to W do
```

```
    c[0, w] = 0
```

```
for i = 1 to n do
```

```
    c[i, 0] = 0
```

```
    for w = 1 to W do
```

```
        if  $w_i \leq w$  then
```

```

    if  $v_i + c[i-1, w-w_i]$  then
         $c[i, w] = v_i + c[i-1, w-w_i]$ 
    else  $c[i, w] = c[i-1, w]$ 
else
     $c[i, w] = c[i-1, w]$ 

```

Example-1

Let us consider that the capacity of the knapsack is $W = 25$ and the items are as shown in the following table.

Item	A	B	C	D
Profit	24	18	18	10
Weight	24	10	10	7

Without considering the profit per unit weight (p_i/w_i), if we apply Greedy approach to solve this problem, first item **A** will be selected as it will contribute maximum profit among all the elements.

After selecting item **A**, no more item will be selected. Hence, for this given set of items total profit is **24**. Whereas, the optimal solution can be achieved by selecting items, **B** and **C**, where the total profit is $18 + 18 = 36$.

Example-2

Instead of selecting the items based on the overall benefit, in this example the items are selected based on ratio p_i/w_i . Let us consider that the capacity of the knapsack is $W = 60$ and the items are as shown in the following table.

Item	A	B	C
Price	100	280	120
Weight	10	40	20
Ratio	10	7	6

Using the Greedy approach, first item **A** is selected. Then, the next item **B** is chosen. Hence, the total profit is $100 + 280 = 380$. However, the optimal solution of this instance can be achieved by selecting items, **B** and **C**, where the total profit is $280 + 120 = 400$.

Hence, it can be concluded that Greedy approach may not give an optimal solution.

To solve 0-1 Knapsack, Dynamic Programming approach is required.

Bellman - ford Algorithm

Bellman-Ford-Algorithm (G, w, s)

for each vertex $v \in G.V$

$v.d := \infty$

$v.\Pi := \text{NIL}$

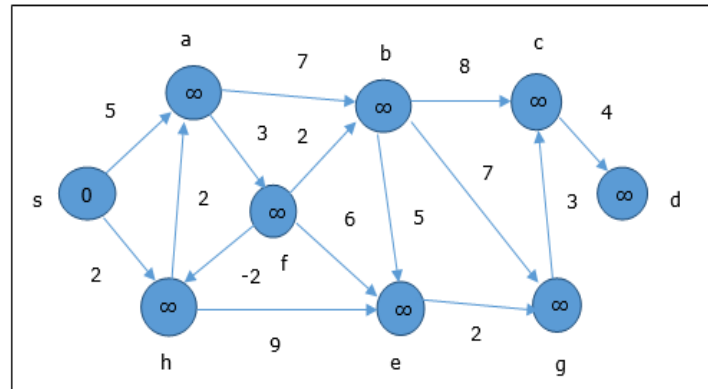
$s.d := 0$

```
for i = 1 to |G.V| - 1
  for each edge (u, v) ∈ G.E
    if v.d > u.d + w(u, v)
      v.d := u.d + w(u, v)
      v.π := u
  for each edge (u, v) ∈ G.E
    if v.d > u.d + w(u, v)
      return FALSE
return TRUE
```

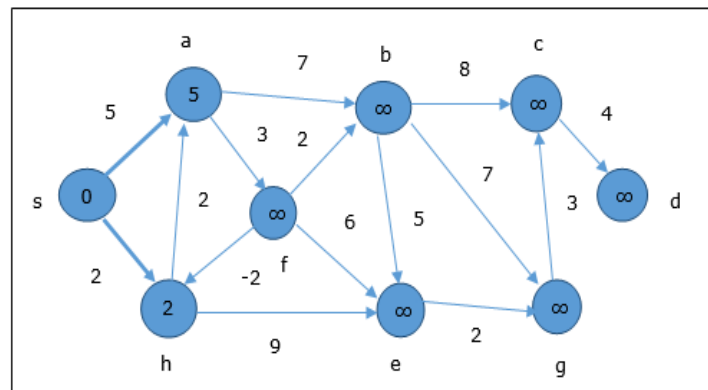
Example

The following example shows how Bellman-Ford algorithm works step by step. This graph has a negative edge but does not have any negative cycle, hence the problem can be solved using this technique.

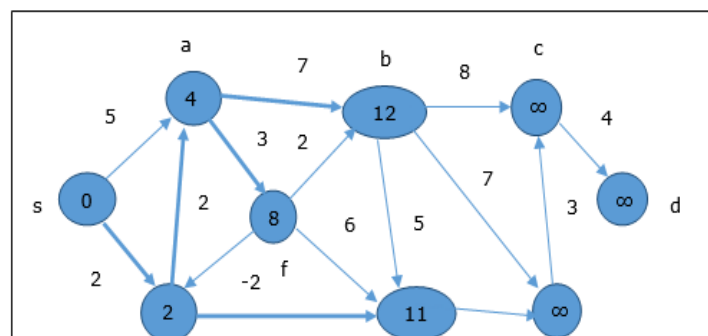
At the time of initialization, all the vertices except the source are marked by ∞ and the source is marked by **0**.



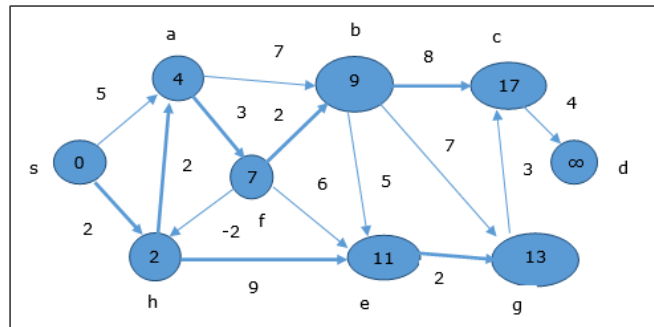
In the first step, all the vertices which are reachable from the source are updated by minimum cost. Hence, vertices **a** and **h** are updated.



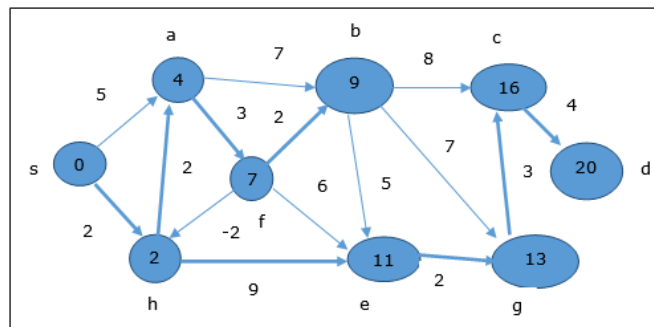
In the next step, vertices **a**, **b**, **f** and **e** are updated.



Following the same logic, in this step vertices **b**, **f**, **c** and **g** are updated.



Here, vertices **c** and **d** are updated.



Hence, the minimum distance between vertex **s** and vertex **d** is **20**.

Based on the predecessor information, the path is $s \rightarrow h \rightarrow e \rightarrow g \rightarrow c \rightarrow d$

Single Source Shortest Path

Dijkstra's Algo

```

Dijkstra's-Algorithm (G, w, s)

for each vertex v ∈ G.V
    v.d := ∞
    v.π := NIL

s.d := 0
S := ∅
Q := G.V

while Q ≠ ∅
    u := Extract-Min (Q)
    S := S ∪ {u}
    for each vertex v ∈ G.adj[u]
        if v.d > u.d + w(u, v)

```

$$v.d := u.d + w(u, v)$$

$$v.\Pi := u$$

Example

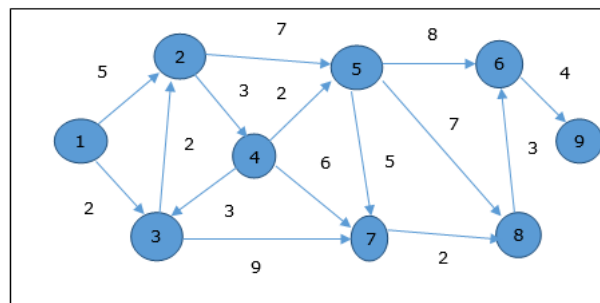
Let us consider vertex **1** and **9** as the start and destination vertex respectively. Initially, all the vertices except the start vertex are marked by ∞ and the start vertex is marked by **0**.

Vertex	Initial	Step1 V_1	Step2 V_3	Step3 V_2	Step4 V_4	Step5 V_5	Step6 V_7	Step7 V_8	Step8 V_6
1	0	0	0	0	0	0	0	0	0
2	∞	5	4	4	4	4	4	4	4
3	∞	2	2	2	2	2	2	2	2
4	∞	∞	∞	7	7	7	7	7	7
5	∞	∞	∞	11	9	9	9	9	9
6	∞	∞	∞	∞	∞	17	17	16	16
7	∞	∞	11	11	11	11	11	11	11
8	∞	∞	∞	∞	∞	16	13	13	13
9	∞	∞	∞	∞	∞	∞	∞	∞	20

Hence, the minimum distance of vertex **9** from vertex **1** is **20**. And the path is

$$1 \rightarrow 3 \rightarrow 7 \rightarrow 8 \rightarrow 6 \rightarrow 9$$

This path is determined based on predecessor information.



Prims Algorithm

Like Kruskal's algorithm, Prim's algorithm is also a Greedy algorithm. It starts with an empty spanning tree. The idea is to maintain two sets of vertices. The first set contains the vertices already included in the MST, the other set contains the vertices not yet included. At every step, it considers all the edges that connect the two sets and picks the minimum weight edge from these edges. After picking the edge, it moves the other endpoint of the edge to the set containing MST.

Prim's Algorithm:

1. Prim's algorithm shares a similarity with the shortest path first algorithms.
2. Prim's algorithm to find minimum cost spanning tree (as Kruskal's algorithm) uses the greedy approach.

Prim's Algo:

Step 1: Select a starting vertex

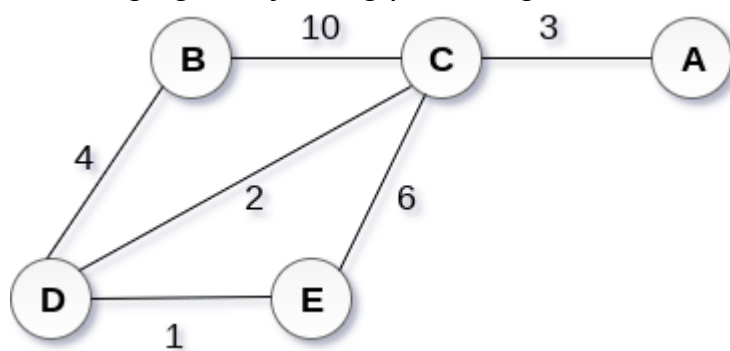
Step 2: Repeat Steps 3 and 4 until there are fringe vertices

Step 3: Select an edge e connecting the tree vertex and fringe vertex that has minimum weight

Step 4: Add the selected edge and the vertex to the minimum spanning tree T
[END OF LOOP]

Step 5: EXIT

Example: Construct a minimum spanning tree of the graph given in the following figure by using prim's algorithm.



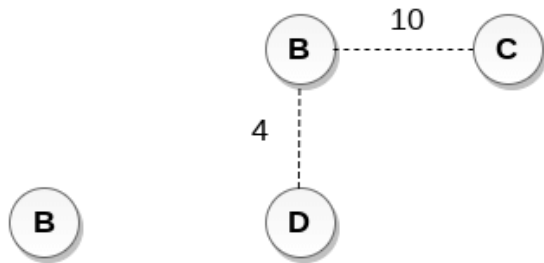
Solution

- Step 1 : Choose a starting vertex B.
- Step 2: Add the vertices that are adjacent to A. the edges that connecting the vertices are shown by dotted lines.
- Step 3: Choose the edge with the minimum weight among all. i.e. BD and add it to MST. Add the adjacent vertices of D i.e. C and E.
- Step 3: Choose the edge with the minimum weight among all. In this case, the edges DE and CD are such edges. Add them to MST and explore the adjacent of C i.e. E and A.
- Step 4: Choose the edge with the minimum weight i.e. CA. We can't choose CE as it would cause cycle in the graph.

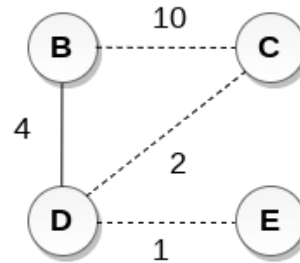
The graph produces in the step 4 is the minimum spanning tree of the graph shown in the above figure.

The cost of MST will be calculated as;

$\text{cost(MST)} = 4 + 2 + 1 + 3 = 10$ units.

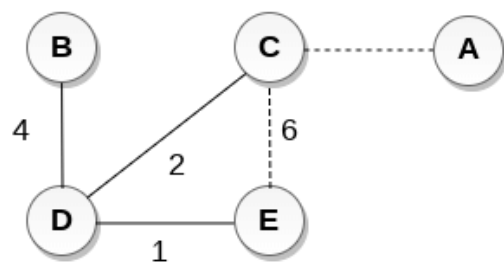


Step 1

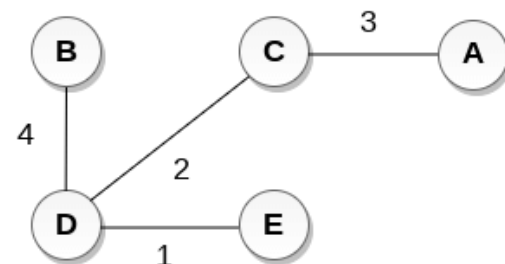


Step 2

Step 3



Step 4



Step 5

Minimum Cost Spanning Tree (MST)

Given a connected and undirected graph, a *spanning tree* of that graph is a subgraph that is a tree and connects all the vertices together. A single graph can have many different spanning trees. A *minimum spanning tree (MST)* or minimum weight spanning tree for a weighted, connected and undirected graph is a spanning tree with weight less than or equal to the weight of every other spanning tree. The weight of a spanning tree is the sum of weights given to each edge of the spanning tree.

Krushkal's Algo

MST- KRUSKAL (G, w)

1. $A \leftarrow \emptyset$

```

2. for each vertex  $v \in V [G]$ 
3. do MAKE - SET ( $v$ )
4. sort the edges of  $E$  into non decreasing order by weight  $w$ 
5. for each edge  $(u, v) \in E$ , taken in non decreasing order by weight
6. do if FIND-SET ( $u$ )  $\neq$  if FIND-SET ( $v$ )
7. then  $A \leftarrow A \cup \{(u, v)\}$ 
8. UNION ( $u, v$ )
9. return  $A$ 

```

1.Kruskal's Algorithm:

Kruskal's Algorithm builds the spanning tree by adding edges one by one into a growing spanning tree. Kruskal's algorithm follows a greedy approach as in each iteration it finds an edge which has least weight and adds it to the growing spanning tree.

1. Sort all the edges in non-decreasing order of their weight.
2. Pick the smallest edge. Check if it forms a cycle with the spanning-tree formed so far. If the cycle is not formed, include this edge. Else, discard it.
3. Repeat step#2 until there are $(V-1)$ edges in the spanning tree.